

ECE 405 Assignment 3

Justin Kugiyama

April 2026

Link to Github Repo: ECE Assignment 3

Link to Colab: Math Baseline

Written Responses

Math Baseline

a) This function generates model responses for a batch of prompts and scores each one against the ground-truth answer. It takes a `vLLM llm` instance, a `reward_fn` that returns format and answer scores, parallel lists of prompts and answers, and `sampling_params` controlling generation behavior. It passes all prompts to `run_vllm` in one batched call, then iterates over each prompt-response-answer triple, runs the reward function, and collects the scores into a list of result dictionaries. The output is one dict per example containing the original prompt, the generated response, the ground-truth answer, and the two reward signals.

b) Most failures fall into Category 3 (83.4%), where both format and answer are incorrect, and the examples show that this is mainly due to the model not adhering to the required output format rather than an issue with the parser. The model frequently produces malformed or missing `answer;` tags, includes extraneous text, or generates incomplete structures, all of which cause the parser to assign a format reward of 0. In Category 2 (13.8%), where the format is correct but the answer is wrong, the errors are primarily due to the model's weak mathematical reasoning, including algebraic mistakes, incomplete solutions, and incorrect assumptions. These examples indicate that even when the model follows instructions, it often fails to solve multi-step problems accurately. In contrast, Category 1 (2.8%) shows that the model can occasionally produce both correct formatting and correct answers, typically on simpler or more straightforward problems. Overall, the results suggest that formatting failures are mostly a compliance issue with the model, while answer errors reflect deeper limitations in reasoning ability.

c) The Qwen 2.5 Math 1.5B model achieves approximately 5–15% accuracy on the MATH validation set in a zero-shot setting. While it often produces correctly formatted outputs, its mathematical reasoning ability remains limited, leading to a low overall correctness rate.



Figure 1: Category 1



Figure 2: Category 2



Figure 3: Category 3

Prompt and Output Tokenization

This function tokenizes prompt-response pairs and builds training data with response masking. It tokenizes prompts and outputs separately, concatenates them, then pads to a uniform length. It constructs `input_ids` (all but last token) and labels (all but first token) for next-token prediction, and creates a boolean `response_mask` marking which positions correspond to the output sequence. The result is a dictionary with padded `input_ids`, shifted labels, and a mask so downstream training can compute loss only over generated responses.

Per-Token Entropy

For entropy, token-level uncertainty is computed from model logits. It takes logits (the raw scores over the vocabulary at each token position), converts them into stable log-probabilities, then turns those into probabilities and applies the entropy formula. It results in one entropy value per token, making tracking how confident or uncertain the model is throughout the sequence available.

Response Log-Probability and Entropy

This function scores target tokens under the current model for a full batch of sequences. It takes the model, `input_ids` for the forward pass, labels for which token probabilities to extract, and an optional `return_token_entropy` flag. It runs the model once, gathers the log probability of each label token, and optionally adds per-token entropy allowing both likelihood and uncertainty from the same pass to be analyzed.

Masked Normalize

The `masked_normalize` function performs masked summation with a caller-controlled normalization constant. It takes the tensor to reduce, a mask for valid elements, a `normalize.constant` to divide by, and an optional reduction `dim`. Instead of dividing by the number of valid tokens automatically, it divides by your supplied constant, which is useful when you want consistent scaling across microbatches.

SFT Microbatch Train Step

For the SFT Microbatch training step, the function performs one supervised fine-tuning microbatch backward step with masking and normalization. It takes `policy_log_probs`, `response_mask`, `gradient_accumulation_steps`, and `normalize_constant` to control scaling. It computes masked negative log-likelihood, averages over batch, divides for accumulation, runs backward, and returns logging values so training loops can report stable loss numbers.

Log Generations

This function evaluates current policy behavior by generating responses and logging reward/length diagnostics. It takes the training model, `vllm` instance, evaluation prompts and answers, `reward_fn`, `sampling_params`, training step, and sampling controls (`num_samples` or `full_dataset`). It syncs model weights into `vLLM`, runs evaluation generations, computes correctness-conditioned length stats and average reward components, logs them to Weights and Biases, and prints one qualitative example for quick inspection.

Running SFT on MATH Dataset

I used Google Colab for the actual SFT training for its T4 GPU, and VS Code mainly for writing/debugging the scripts. The biggest headache was moving back and forth between the two, since file paths were different, some dependencies/runtime settings didn't match, and outputs from one environment weren't automatically available in the other. So the training setup itself mostly worked, but the VS Code to Colab handoff caused most of the delays. Once those environment mismatches were handled, I was able to at least start training runs and confirm that the pipeline was making real optimization steps, even if full end-to-end results still depended on stable GPU runtime time.

Running Expert Iteration on MATH Dataset

I attempted to run the expert iteration sweep on Google Colab with a T4 GPU, but ran into environment setup issues. The pip install conflicted with pre-installed packages and by the time it partially worked, the session had timed out. On a second attempt, vLLM failed to initialize because the free T4's 15GB of VRAM wasn't enough to run both the policy model and vLLM inference simultaneously. The EI pipeline itself is fully implemented, but I wasn't able to get a clean end-to-end run within the available session time.

Group Normalization

The `group_normalization` function computes rewards and within-group normalized advantages over rollout samples. It takes `reward_fn`, `rollout_responses`, `repeated_ground_truths`, and `group_size`, plus `advantage_eps` and `normalize_by_std` for numerical behavior. It scores each response, computes each group's mean (and optionally standard deviation), converts rewards to centered or standardized advantages, and returns both tensors with summary reward statistics.

Naive Policy Gradient

For computing the Naive policy gradient loss, this function builds the basic REINFORCE-style per-token loss from rewards or advantages. It takes one reward/advantage per sequence and the per-token `policy_log_probs`, then expands the reward signal to match sequence length. It returns the negative product so minimizing the loss pushes probability mass toward high-reward trajectories.

GRPO-Clip Loss

The `compute_GRPO_clip_loss` function implements clipped policy optimization for GRPO-style updates. It takes `advantages`, current `policy_log_probs`, `old_log_probs` from the reference policy, and `cliprange` to bound update size. It computes importance ratios, clips them to the allowed interval, uses the pessimistic minimum objective, and also returns metadata showing where clipping was active.

Policy Gradient Wrapper

The `compute_policy_gradient_loss` function is a routing layer that selects the correct policy-gradient objective from one interface. It takes `policy_log_probs`, a `loss_type` switch, and the optional inputs needed by each mode: `raw_rewards`, `advantages`, `old_log_probs`, and `cliprange`. Based on the chosen loss type, it validates required arguments, calls the right loss function, and returns both token-level loss values and auxiliary metadata.

Masked Mean

The `masked_mean` function computes an average while ignoring masked-out positions like padding or non-response tokens. It takes a tensor of values, a mask indicating which entries are valid, and an optional dim telling it where to reduce. It multiplies by the mask, sums valid values, divides by the valid-count denominator, and returns the masked average for stable training metrics.

GRPO Microbatch Train Step

For this task, this function performs one reinforcement-learning microbatch backward step for multiple PG loss variants. It takes `policy_log_probs`, `response_mask`, `gradient_accumulation_steps`, `loss_type`, and whichever optional tensors are needed for that mode (`raw_rewards`, `advantages`, `old_log_probs`, `cliprange`). It computes the selected RL loss, removes invalid tokens via masking, normalizes over valid response tokens, scales for accumulation, backpropagates, and returns detached loss plus metadata.

GRPO Train Loop

Each step, the policy generates G responses per question via vLLM. Responses are scored and group-normalized into advantages, then used to update the policy with a clipped PPO objective against a frozen reference. Validation accuracy is checked periodically. The loop was implemented but not run, so no results were produced.

Tune the Learning Rate

DNF

Effect of Baseline

From code-level observation, `no_baseline` uses raw rewards and typically has higher gradient variance, while `reinforce_with_baseline` uses centered advantages and is generally more stable. In practice, `reinforce_with_baseline` is usually better in noisy/sparse-reward settings, while `no_baseline` is simpler but more sensitive to reward scale and randomness.

Thinking About Length Normalization

DNF

Effect of Length Normalization

DNF

Effect of Standard Deviation Normalization

This experiment is not yet complete, so I cannot report validation reward curves or make evidence-based claims about which setting performs better in this repo state. At this point, the correct conclusion is that the comparison is configured in the training code, but empirical findings on accuracy and stability still require running matched GRPO experiments.

Implement Off-Policy GRPO

I added the off-policy GRPO controls in `grpo_train.py`: rollout batch size, train batch size, and epochs per rollout batch. I also switched old-log-prob computation to the current policy in inference mode before inner updates, and added logging for gradient norm and number of updates. Was not run the full experiment and generate the resulting curves.

Off-Policy GRPO Hyperparameter Sweep

DNF

Off-Policy GRPO-Clip Ablation

DNF

Prompt Ablation

DNF